P2319R1
Prevent path presentation problems

Published Proposal, 2024-09-14

Author:
Victor Zverovich

Audience:
SG16, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

"We are stuck with technology when what we really want is just stuff that works." — Douglas Adams

§ 1. Introduction

[P2845] made it possible to format and print `std::filesystem::path` with correct handling of Unicode. Unfortunately, some common path accessors still exhibit broken behavior, which results in

mojibake and data loss. This paper proposes deprecating these accessors, making the path API more reliable and eliminating a common source of bugs.

§ 2. Changes since R0

- Removed `system_string` and `display_string` per SG16 feedback focusing just on deprecating broken accessors.

§ 3. Problem

Consider the following example:

```
std::filesystem::path p(L"Выявы"); // Выявы is Images in Belarusian.
std::cout << p << std::endl;
std::cout << p.string() << std::endl;
```

Even if all code pages and localization settings are set to Belarusian and both the source and literal encodings are UTF-8, this still results in mojibake on Windows:

```
"┬√ т√"
┬√ т√
```

Unfortunately, we cannot change the behavior of iostreams but at least the new facilities such as `std::format` and `std::print` correctly handle Unicode in paths. For example:

```
std::filesystem::path p(L"Выявы");
std::print("{}\n", p);
```

prints

```
Выявы
```

However, the `string()` accessor still exhibits the broken behavior, e.g.

```
std::filesystem::path p(L"Выявы");
```

```
std::print("{}\n", p.string());
```

prints

```
?????
```

The reason for this is that `std::filesystem::path::string()` transcodes the path into the native encoding ([fs.path.type.cvt]) defined as:

> The native encoding of an ordinary character string is the operating system dependent current encoding for pathnames ([fs.class.path]).

It is neither the literal encoding nor a locale encoding, and transcoding is usually lossy, which makes it almost never what you want. For example:

```
std::filesystem::path p(L"Obrázky");
std::string s = p.string();
```

throws `std::runtime_error` with the message "unknown error" on the same system which is a terrible user experience.

The string can be passed to system-specific APIs that accept paths provided that the system encoding hasn't changed in the meantime. But even this use case is limited because the transcoding is lossy, and it's better to use an equivalent `std::filesystem` API or `native()` instead.

On Windows, the native encoding is effectively the Active Code Page (ACP), which is separate from the console code page. This is why paths often cannot be correctly displayed. Even Windows documentation ([CODE-PAGES]) cautions against using code pages:

> New Windows applications should use Unicode to avoid the inconsistencies of varied code pages and for ease of localization.

Encoding bugs are even present in standard library implementations, see e.g. [LWG4087], where a path in the "native" encoding is incorrectly combined with text in literal and potentially other encodings when constructing an exception message.

Moreover, the result of `string()` is affected by a runtime setting and may work in a test environment but easily break after deployment. This is similar to one of the problems with `std::locale` but worse because in this case C++ doesn't even provide a way to set or query the encoding. It dispropor-

tionately affects non-English C++ users making the language not as attractive for writing internationalized and localized software.

§ 4. Proposal

To summarize, `std::filesystem::path::string()` has the following problems:

- It uses encoding that is generally incompatible with nearly all standard text processing and I/O facilities including iostreams, `std::format` and `std::print`.
- It is extremely error-prone, causing easy to miss transcoding issues that may arise after the program is deployed in a different environment or after a runtime configuration change.
- It makes writing portable code hard because the issues may not be obvious on POSIX platforms where `string()` is just an inefficient equivalent of `native()` with extra memory allocation and copy.

The current paper proposes deprecating the `std::filesystem::path::string()` and `std::filesystem::path::generic_string()`, which has the same problems, overloads for `char`.

There is usually a better way to accomplish the same task with non-legacy APIs, e.g. using the lossless `std::filesystem::remove` that takes a path object instead of `std::remove`:

```
std::filesystem::remove(p); // Lossless, portable and more efficient.
```

Ideally, `std::remove` should be deprecated but this is out of scope of the current paper.

And for lossless display, these accessors can be replaced with formatting `path` using new facilities such as `std::format` or `std::print`.

§ 5. Wording

Modify [https://eel.is/c++draft/fs.class.path.general]:

```
...

// [fs.path.native.obs], native format observers
const string_type& native() const noexcept;
```

```
const value_type*  c_str() const noexcept;
operator string_type() const;

template<class EcharT, class traits = char_traits<EcharT>,
          class Allocator = allocator<EcharT>>
  basic_string<EcharT, traits, Allocator>
    string(const Allocator& a = Allocator()) const;
std::string    string() const;
std::wstring   wstring() const;
std::u8string  u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;

// [fs.path.generic.obs], generic format observers
template<class EcharT, class traits = char_traits<EcharT>,
          class Allocator = allocator<EcharT>>
  basic_string<EcharT, traits, Allocator>
    generic_string(const Allocator& a = Allocator()) const;
std::string    generic_string() const;
std::wstring   generic_wstring() const;
std::u8string  generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;

...
```

Modify [fs.path.native.obs]:

...

```
std::string string() const;
std::wstring wstring() const;
std::u8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;
```

Returns: native().

Remarks: Conversion, if any, is performed as specified by [fs.path.cvt].

Modify [fs.path.generic.obs]:

...

```
~~std::string generic_string() const;~~
std::wstring generic_wstring() const;
std::u8string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
```

Returns: The pathname in the generic format.

Remarks: Conversion, if any, is specified by [fs.path.cvt].

Add a new subclause in Annex D:

Deprecated filesystem path format observers [depr.fs.path.obs]

The following `filesystem::path` member is defined in addition to those specified in [fs.path.native.obs]:

```
std::string string() const;
```

Returns: native().

Remarks: Conversion, if any, is performed as specified by [fs.path.cvt].

The following `filesystem::path` member is defined in addition to those specified in [fs.path.generic.obs]:

```
std::string generic_string() const;
```

Returns: The pathname in the generic format.

Remarks: Conversion, if any, is specified by [fs.path.cvt].

§ References

§ Informative References

[CODE-PAGES]

Windows App Development / Code Pages. URL: https://learn.microsoft.com/en-us/windows/win32/intl/code-pages

[LWG4087]

LWG Issue 4087: Standard exception messages have unspecified encoding. URL: https://cplusplus.github.io/LWG/issue4087

[P2845]

Victor Zverovich. *Formatting of std::filesystem::path*. URL: https://wg21.link/p2845